

Artefactos técnicos y documentación automatizada

Documentación as-a-code

1. Introducción

Imaginá este escenario típico en un equipo de desarrollo: el programador termina una funcionalidad nueva, la sube al repositorio y la documentación queda en un Google Doc que nadie actualiza. Seis meses después, otro desarrollador lee esa documentación y está completamente desactualizada. Nadie sabe cuál es la versión correcta, quién la modificó por última vez ni por qué se tomó cierta decisión de diseño. Este fenómeno tiene nombre: se llama *documentation drift* (desviación de la documentación), y es uno de los problemas más comunes en equipos de software.

El problema central: Cuando la documentación vive separada del código (en wikis, Word, PDFs o Google Docs), tiende a quedar desactualizada rápidamente. Nadie la revisa, nadie la actualiza y nadie confía en ella. Docs as Code surge como solución a este problema.

2. ¿Qué es Documentación as-a-code?

Documentación as-a-Code (Docs as Code) es un enfoque moderno que consiste en tratar la documentación técnica con las mismas herramientas, procesos y estándares que se usan para el código fuente. En lugar de escribir los documentos en Word o Google Docs, se escriben en Markdown (un formato de texto simple), se guardan en un repositorio Git, se revisan mediante pull requests y se publican automáticamente mediante pipelines de CI/CD.

2.1 Diferencia con la documentación tradicional

Aspecto	Documentación tradicional	Docs as Code
¿Dónde se escribe?	Word, Google Docs, Confluence, wikis separadas	Archivos Markdown en el mismo repositorio Git del proyecto
¿Quién la mantiene?	Generalmente una sola persona (redactor técnico)	Todo el equipo: devs, QA, tech writers
Control de cambios	Manejo manual de versiones (doc_v2_final_FINAL.docx)	Git registra cada cambio con autor, fecha y motivo
Revisión	Por email, comentarios en PDF o reuniones	Pull request con revisión línea por línea, igual que el código
Publicación	Manual: exportar, subir, compartir enlace	Automática: el pipeline la publica en cada merge
Actualización	Suele quedar desactualizada porque es un paso separado	Se actualiza junto con el código en el mismo commit

3. Los 4 pilares de docs as-a-code

Docs as a Code se apoya en cuatro componentes fundamentales que trabajan juntos para mantener la documentación actualizada, colaborativa y automatizada:

3.1 Pilar 1 – Control de versiones con git

Git no solo sirve para versionar código; funciona igual de bien para documentos de texto. Al guardar la documentación en Git se obtiene:

- Historial completo: se puede ver quién escribió cada línea, cuándo y por qué.
- Ramas (branches): se puede trabajar en una sección nueva sin afectar la documentación vigente.
- Revertir cambios: si se comete un error, se vuelve a cualquier versión anterior en segundos.
- Trazabilidad: cuando algo está mal en la documentación, git blame muestra exactamente quién lo cambió.

3.2 Pilar 2 – Formato de texto plano (Markdown)

Markdown permite escribir documentación utilizando caracteres simples de texto que luego se transforma automáticamente en contenido formateado, facilitando su mantenimiento y versionado junto al código. Es el formato estándar en Docs as-a-Code porque:

- Es legible sin renderizar: se entiende incluso leyendo el archivo crudo.
- Es texto plano: Git puede comparar diferencias línea por línea, al igual que el código.
- Es universal: GitHub, GitLab, Notion, Obsidian y muchas otras herramientas lo soportan.
- Es fácil de aprender: la curva de aprendizaje es mínima.

3.3 Pilar 3 – Revisión colaborativa (Pull Requests)

En Docs-as-Code, cualquier cambio en la documentación pasa por un pull request (también llamado merge request en GitLab), igual que el código. Esto significa que:

- Otro miembro del equipo debe revisar y aprobar el cambio antes de que se incorpore.
- Se pueden hacer comentarios línea por línea sobre partes específicas del texto.
- Se controla que la documentación sea precisa y esté bien escrita antes de publicarse.
- Se distribuye el conocimiento sobre el proyecto entre todos los que revisan.

3.4 Pilar 4 – Automatización CI/CD

CI/CD significa Integración Continua y Despliegue Continuo. En el contexto de Docs as Code, es el mecanismo que hace que la documentación se valide y publique automáticamente sin intervención manual.

Paso del pipeline	Qué hace	Ejemplo
CI — Integración	Cuando se sube un cambio, el sistema ejecuta validaciones automáticas	Verifica ortografía, links rotos y formato Markdown
Build	Compila los archivos Markdown en un sitio web estático	MkDocs convierte los .md en páginas HTML
CD — Despliegue	Publica el sitio generado automáticamente	GitHub Pages o Netlify sirven la documentación actualizada

Resultado final: Cada vez que un desarrollador hace un commit que modifica el código y su documentación correspondiente, el pipeline valida, construye y publica la documentación actualizada de forma completamente automática. Sin pasos manuales, sin olvidarse de actualizar nada.

4. Diagramas como código

4.1 ¿Qué significa "diagrama como código"?

Crear un diagrama de arquitectura de manera tradicional implica abrir una herramienta gráfica (draw.io, Lucidchart, PowerPoint), arrastrar cajas y flechas, exportar la imagen y subirla al repositorio. Cada vez que hay un cambio en el sistema, hay que repetir todo ese proceso manualmente.

Diagrama como código propone algo completamente diferente: en vez de dibujar el diagrama visualmente, se describe su estructura con texto, y una herramienta lo convierte automáticamente en un gráfico. El diagrama existe como un archivo de texto que se puede versionar en Git igual que cualquier otro artefacto del proyecto.

Aspecto	Diagrama tradicional (imagen)	Diagrama como código (texto)
Formato	PNG, SVG, PDF (binario)	Texto plano (.md, .mmd, .puml)
Versionado	Git no puede comparar diferencias	Git muestra exactamente qué cambió
Edición	Requiere herramienta gráfica	Cualquier editor de texto
Actualización	Manual, proceso largo	Cambiar una línea de texto
Colaboración	Difícil de revisar en PR	Revisable línea a línea en PR
Integración con docs	Imagen externa linkada	Inline en el propio Markdown

4.2 Mermaid.js: la herramienta principal

Mermaid.js es una biblioteca JavaScript de código abierto que convierte descripciones de texto en diagramas visuales. Su sintaxis está inspirada en Markdown: es simple, legible y no requiere aprender nada complejo. Fue creada por Knut Sveidqvist con el objetivo de facilitar la documentación. Actualmente es la herramienta de diagramación más popular en el ecosistema Docs as Code.

Características clave de Mermaid:

- **Soporte nativo en GitHub y GitLab:** los diagramas se renderizan automáticamente en los archivos README.md sin plugins adicionales.
- **Integración con editores:** VS Code, IntelliJ y otros IDEs tienen plugins que muestran el diagrama en tiempo real mientras se escribe.
- **Editor online gratuito:** mermaid.live permite crear y previsualizar diagramas sin instalar nada.
- **Exportación:** los diagramas se pueden exportar en PNG, SVG u otros formatos.
- **Código abierto:** gratuito, con comunidad activa y en constante desarrollo.

Ejemplo diagrama de flujo: proceso de login de un usuario

```
```mermaid flowchart TD
 A([Inicio]) --> B[Usuario ingresa email y contraseña]
 B --> C{¿Credenciales válidas?}
 C -->|Sí| D[Generar token JWT]
 C -->|No| E[Mostrar error]
 D --> F([Acceso al sistema])
 E --> B
  ```
```

*En este ejemplo: **TD** significa Top-Down (de arriba hacia abajo). Los corchetes **[]** son rectángulos (pasos). Las llaves **{ }** son rombos (decisiones). Los paréntesis dobles **([])** son óvalos (inicio/fin).*

5. IA para generar diagramas y documentación

5.1 Cómo la IA acelera la documentación técnica

Los modelos de inteligencia artificial generativa (como ChatGPT, Claude u otros) pueden generar código Mermaid y documentación técnica a partir de descripciones en lenguaje natural. Esto significa que un desarrollador puede explicar su sistema con palabras simples y obtener un diagrama listo para usar. Por ejemplo, si le pedís a una IA: "Generá un diagrama de secuencia Mermaid para el proceso de registro de usuario: el frontend envía nombre, email y

contraseña al backend, que valida los datos, guarda el usuario en la base de datos y devuelve un token de confirmación", la IA produce el código del diagrama completo, listo para pegarlo en el README.

Importante: La IA genera un borrador que hay que revisar y ajustar. No reemplaza el criterio del programador; lo complementa eliminando el tiempo que se pierde en recordar la sintaxis o estructurar el diagrama desde cero.

5.2 Flujo de trabajo con IA

1. Describir el sistema

El desarrollador explica en lenguaje natural qué quiere documentar: un proceso, una arquitectura, las relaciones entre clases, etc.

2. La IA genera el código

El modelo produce el código Mermaid (o PlantUML, o texto Markdown) correspondiente a la descripción

3. Revisar y ajustar

El desarrollador verifica que el diagrama sea correcto y solicita cambios o mejoras a la IA de ser necesario.

4. Incorporar al repositorio

El código del diagrama se guarda en el archivo Markdown del proyecto y se sube como parte de un commit, como cualquier otro artefacto.

6. ventajas del enfoque docs as-a-code

Documentación siempre actualizada: Al vivir junto al código y actualizarse en el mismo commit, es mucho menos probable que quede desactualizada.

Versionado y trazabilidad: La documentación se versiona igual que el código. Lo que permite ver que cambios se hicieron, quien los realizó, recuperar versiones anteriores. Esto mejora el control del proyecto.

Colaboración estructurada: El proceso de pull request obliga a revisar la documentación antes de publicarla, igual que se revisa el código.

Automatización total: La publicación es automática; no hay que recordar subir nada manualmente ni exportar archivos.